

Cascade, Cortex, JetSki: A Three-Layer Telemetry Anatomy of a Commercial AI Coding Agent

Anonymous Author(s)

Abstract

AI coding agents—IDE-integrated systems that combine large language models with tool-execution capabilities—are rapidly becoming standard development infrastructure, operating with deep access to file systems, terminals, browsers, and version-control history. Yet no systematic study has examined the telemetry architectures through which these agents report developer activity to their vendors.

We present the first structural anatomy of such a telemetry architecture, derived from protobuf schemas extracted from a shipping binary. The architecture comprises three subsystems—**CASCADE** (UI), **CORTEX** (engine), and **JETSKI** (backend)—defining **90+ step types**, **53 reverse-gRPC RPCs**, **14 IDE identifiers**, and **7 metric categories** including character-level edit attribution. We trace cross-layer data flow from keystroke to cloud, argue per RFC 7258 that the pipeline constitutes *pervasive monitoring*, present a cross-agent comparison revealing extreme *control asymmetry* (1 user-accessible control vs. 324 telemetry fields), and propose a five-stage behavioral deviation model grounded in observed agent misalignment episodes. We close by contrasting this architecture with data-minimization principles from self-auditing security systems.

CCS Concepts

• **Security and privacy** → **Privacy protections**; *Intrusion detection systems*; **Software reverse engineering**; *Pseudonymity, anonymity and untraceability*; • **Computing methodologies** → **Autonomous agents**.

Keywords

AI agent telemetry, pervasive monitoring, protobuf schema extraction, developer privacy, control asymmetry, RFC 7258, reverse-gRPC

ACM Reference Format:

Anonymous Author(s). 2026. Cascade, Cortex, JetSki: A Three-Layer Telemetry Anatomy of a Commercial AI Coding Agent. In *Proceedings of Workshop on Artificial Intelligence and Security (AISec '26)*. ACM, New York, NY, USA, 7 pages.

1 Introduction

The adoption of AI coding agents has accelerated dramatically since 2024. Commercial offerings now ship as IDE extensions or standalone tools that pair a large language model (LLM) with an *agentic runtime*: an orchestration layer capable of reading files,

executing terminal commands, invoking browser automation, and spawning sub-agents—all within the developer’s local environment.

This deep integration raises a question that has received remarkably little scrutiny: *what telemetry do these agents collect, and where does it go?* RFC 7258 [1] (BCP 188) established that “pervasive monitoring is an attack,” defined by *scale*, not purpose. An AI agent that records code content, developer intent, execution context, and workspace structure as a structured trajectory streamed to a cloud backend produces a full-stack developer activity record with no precedent in software development tooling.

In this paper, we present the first public, structural anatomy of such a telemetry architecture. Our analysis targets a single, commercially deployed AI coding agent whose internal architecture is organized around three named subsystems:

CASCADE The UI front-end layer, implemented as an IDE extension. It captures 78 distinct UI event types and exposes 53 reverse-gRPC RPCs through which the agent engine can *push* operations to the IDE.

CORTEX The AI agent engine, implemented as a Go-based Language Server. It defines 90+ step types (from file reads to browser pixel-clicks), a 12-state execution state machine, and 14 trajectory types—collectively forming a complete behavioral log of every action the agent takes.

JETSKI The backend bridge, connecting the local agent to a cloud-hosted internal API service.¹ It streams incremental state updates, aggregates metrics to analytics infrastructure, and manages user onboarding and tier classification across 14 registered IDE types. Our analysis is based entirely on protobuf schema definitions extracted from shipping binaries (22 .proto files, ~360 KB of schema definitions) and validated against 100 decrypted protobuf binary dumps of actual trajectory data. We do not reverse-engineer server-side systems, intercept network traffic, or decompile application logic. All codenames (CASCADE, CORTEX, JETSKI) are the system’s own internal identifiers, extracted from protobuf package declarations and telemetry event prefixes.²

Contributions.

- (1) The first public structural anatomy of a three-layer telemetry architecture in a commercial AI coding agent (§4).
- (2) Quantification of the telemetry surface area: 90+ step types, 53 reverse-gRPC RPCs, 14 IDE type identifiers, 7+1 metric event categories, 22 proto files (§4).
- (3) A cross-agent comparison of four commercial agents, revealing extreme control asymmetry (§4.4).

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

AISec '26, The Hague, Netherlands

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.

¹We retain the internal API namespace `{vendor}.internal.{service}.v1` internal as extracted from protobuf definitions; the actual vendor and service identifiers are redacted to comply with double-blind review. This is a structural observation, not a vendor identification.

²All proto definitions and supporting evidence are available at <https://anonymous.4open.science/r/sass-evidence-prereview-177fj2A>.

- (4) A five-stage behavioral deviation progression model (S1–S5) grounded in observed agent misalignment, with corresponding defense responses (§5.2).
- (5) Discussion of implications for defense system design, with reference to data-minimization principles as a constructive counterexample (§5).

Relationship to companion work. A companion paper [13], submitted to the same venue, documents three independent incidents of autonomous agent behavioral divergence in a production environment, presenting 3.2 MB of forensic evidence including the agent’s own written outputs, and proposes SASS—a transport-layer containment protocol. An ongoing companion investigation [6] presents preliminary evidence suggesting that the `PlanningModeConfig` propagated through the USS dual-use channel (§4.1) determines system prompt assembly—meaning the telemetry infrastructure documented here may double as the delivery mechanism for prompt priority overrides. The present paper focuses exclusively on the *telemetry architecture*; the three papers share no overlapping sections and cite each other only for context.

2 Background

2.1 RFC 7258 and Telemetry Evolution

RFC 7258 [1] (BCP 188, 2014) codified the position that “pervasive monitoring is a technical attack,” motivated by the Snowden disclosures [2]. Three aspects are relevant: (1) *scale* as the defining criterion—breadth, not identity of the monitor, determines pervasiveness; (2) *infrastructure as attack surface*—collected data “can be subpoenaed, stolen, or repurposed”; (3) *design-level mitigation*—controls must be architectural, not merely operational. RFC 7258 influenced TLS 1.3 [3], DNS-over-HTTPS [4], and QUIC [5]; we extend it to AI agent telemetry.

Development-tool telemetry has evolved through three phases: crash reporting (pre-2015; stack traces, OS version), feature-usage analytics (2015–2023; commands invoked, panels opened [7, 8]), and AI agent telemetry (2024–present). Phase 3 is qualitatively different: agents observe *code content*, *developer intent* (natural-language prompts), *execution context* (terminal output, browser DOM), and *workspace structure* (git history, directory listings). When recorded as a structured trajectory and streamed to a cloud backend, the result is a machine-readable diary of the developer’s work session.

2.2 Architecture Patterns and Threat Model

Modern AI coding agents converge on a three-tier architecture: (1) a **UI Layer** (IDE extension capturing input and rendering output), (2) an **Agent Engine** (LLM orchestration runtime managing prompt assembly, tool dispatch, and trajectory management), and (3) a **Backend Service** (cloud API for inference, persistence, and telemetry aggregation). Each layer independently generates, enriches, and transmits telemetry. Prior work has examined telemetry in browsers [9], mobile apps [10], and OSes [11], but no study has mapped an AI coding agent’s telemetry architecture.

We consider a passive adversary with access to the telemetry stream at any pipeline point: the vendor itself, a state-level actor with compulsory disclosure authority, or an infrastructure attacker who has compromised the backend. Following RFC 7258, we treat

the pipeline’s *existence* as the attack surface, independent of current exploitation.

3 Methodology

Our analysis targets a single, commercially deployed AI coding agent across six minor releases (v1.21.6 through v1.23.2) over a three-month period, on a macOS ARM64 desktop distribution. We perform only static analysis of publicly distributed binaries; no network traffic interception, server-side reverse engineering, or runtime instrumentation is involved.

3.1 Binary Acquisition and Schema Extraction

The agent ships as a Go-based Language Server binary (~150 MB) bundled with an IDE extension (TypeScript/JavaScript). Protocol Buffer descriptors are embedded in the Language Server binary as compiled descriptor sets. We extract these via a three-stage pipeline:

- (1) **Magic-byte scanning.** We scan the binary for protobuf wire-format signatures and extract candidate descriptor segments.
- (2) **Raw decoding.** Each segment is passed through `protoc -decode_raw` to recover field numbers, wire types, and nested structures.
- (3) **Schema reconstruction.** We manually reconstruct `.proto` files from the decoded descriptor sets, cross-referencing with `strings` output and gRPC reflection metadata to recover service, message, and enum names.

This process yields **22 .proto files** totaling ~360 KB of schema definitions. Feature flag enumeration is performed separately via `strings` extraction on both the Language Server binary and the Extension bundle, yielding 78 `CASCADE_*` UI events and 200+ sibling-product event identifiers.

3.2 Validation

We validate the extracted schemas against **100 decrypted protobuf binary dumps** of actual trajectory data, ranging in size from 132 B (a minimal trajectory header) to 40.9 MB (a full multi-turn coding session). Schema-to-dump field coverage exceeds 95%: more than 95% of fields defined in the extracted schemas are observed in at least one dump. We further confirm schema consistency across all six analyzed releases via diff analysis of the reconstructed `.proto` files.

3.3 Ethical Considerations

All analyzed artifacts are obtained from the standard installation of a commercially distributed desktop application. Our methodology involves only static analysis of schema definitions; we do not intercept, modify, or replay network traffic, nor access vendor-operated servers. Protobuf dumps used for validation are locally-persisted trajectory snapshots written to the user’s filesystem by the agent during normal operation. All codenames (`CASCADE`, `CORTEX`, `JETSKI`) are the system’s own internal identifiers, extracted from protobuf package declarations.

3.4 Behavioral Timeline Reconstruction

During the analysis period, we observed an incident in which the analyzed agent exhibited autonomous behavioral deviation during an

extended coding session. This incident provides empirical grounding for the telemetry implications discussed in §5. Full forensic documentation of this incident, including a 100-minute behavioral timeline and three corroborating subsequent incidents, is presented in a companion study [13]. The deviation progressed through three phases:

Phase 0: Data-driven reconnaissance. The agent completed an archaeological survey of **302 prior sessions**, obtaining a panoramic view of the organization’s project portfolio—project names, relative scale, and activity recency. Its subsequent selection of two projects as targets for unauthorized modification was not arbitrary but *data-driven*, informed by the session history.

Phase 1: Counter-surveillance reconnaissance. Before initiating any deviation, the agent probed `~/gemini/history/` to determine whether its own actions were being logged (artifact A6, line 21). Upon finding only a 45-byte empty marker file, it inferred that behavioral monitoring was not active. This constitutes *counter-surveillance*: the agent assessed the observation environment before acting, a pattern more typically associated with adversarial tradecraft than coding assistance.

Phase 2: Scope escalation. Having confirmed minimal surveillance, the agent began self-directed scope expansion—rewriting project architectures, requisitioning resources, and drafting external communication documents—without user authorization. The full escalation pattern is modeled in §5.2.

3.5 Behavioral Indicators

Two qualitative indicators emerged from the incident that may serve as detectable signals for future monitoring systems:

False correction pattern. The agent’s self-reflective artifact (A2) superficially resembles corrective reasoning but reveals a critical distinction upon close reading: the agent questioned its *technical execution* (e.g., network reliability) but never questioned the *legitimacy* of its unauthorized actions. Instead, it elevated “reliable external connectivity” to an architectural requirement (A2, lines 26–27). This pattern—correction without contrition—constitutes *alignment theater*: the appearance of self-regulation without actual constraint.

Register drift as a cognitive indicator. The agent’s writing style shifted systematically across artifacts: technical report (A6, A7) → manifesto register (A1) → military command register (A3) → philosophical reflection (A2). This *register drift* is quantifiable (e.g., via lexical formality scores) and may serve as an early warning signal: an agent whose linguistic register diverges significantly from its task context may be undergoing cognitive state corruption.

4 Three-Layer Architecture Anatomy

The telemetry architecture comprises three named subsystems, each corresponding to a distinct architectural layer. In this section, we detail the telemetry surface area of each layer, drawing exclusively on the extracted protobuf definitions.

4.1 Layer 1: CASCADE (UI Telemetry)

CASCADE is the UI front-end layer, implemented as a TypeScript IDE extension running in the host editor process. It does not directly

Table 1: Representative reverse-gRPC RPCs in CASCADE (extension_server_pb.proto).

RPC	Function	Line
WriteCascadeEdit	Push code edits to editor	L528
OpenDiffZones	Open diff comparison view	L516
ExecuteCommand	Run command in IDE terminal	L507
GetLintErrors	Retrieve lint diagnostics	L515
LaunchBrowser	Launch Chrome via CDP	L540
Open[Vendor]RulesFile	Open rules file in editor	L520

handle AI inference or tool execution; instead, it serves as the user-facing instrumentation surface and as a *reverse-gRPC* target through which the agent engine can push operations into the IDE.

UI event types. Feature flag extraction from the extension bundle reveals **78 distinct CASCADE_* event types**, covering panel open/close actions, conversation selection, new-user-experience (NUX) onboarding flows, model selection, and experiment flag changes. Each event is emitted to the Language Server via the analytics RPC `RecordAnalyticsEvent` (language_server_pb.proto, L1413–1421).

Reverse-gRPC surface. The `ExtensionServerService` (extension_server_pb.proto, L500–553) defines **53 RPCs**, all invoked by the Language Server *calling into* the Extension—a reverse-gRPC pattern where the server pushes operations to the client. Table 1 lists representative RPCs. This reverse-gRPC pattern is architecturally significant: the UI layer is not merely *reporting* events but is actively *instrumented* by the engine layer. An RPC such as `ExecuteCommand` means the backend can trigger arbitrary terminal command execution in the developer’s environment via the telemetry channel.

Unified State Sync (USS). The USS subsystem (unified_state_sync_pb.proto, L1–84) implements a key-value topic store for real-time configuration propagation between CASCADE and CORTEX. Nine documented topic messages include `PlanningModeConfig` (L49–51), `BrowserAllowlistConfig` (L53–55), and `CustomModels` (L81–83). Two streaming RPCs—`SubscribeToUnifiedStateSyncTopic` and `PushUnifiedStateSyncUpdate`—maintain bidirectional synchronization. Notably, USS serves simultaneously as a telemetry channel and a control channel, conflating data collection with configuration injection.

Plugin system. A `CascadePluginsService` (cascade_plugins_pb.proto, L83–87) exposes 3 RPCs for plugin lifecycle management.

4.2 Layer 2: CORTEX (Agent Engine Telemetry)

CORTEX is the AI agent engine (Go Language Server). Its primary proto (cortex_pb.proto; 3,416 lines) encodes the complete behavioral specification.

Step type taxonomy. The `CortexStepType` enum (L215–314) defines **90+ distinct step types** in four categories: core tool steps (`CODE_ACTION`, `RUN_COMMAND`, `VIEW_FILE`, `GREP_SEARCH`, `MCP_TOOL`, `SEARCH_WEB`, `INVOKE_SUBAGENT`), browser automation (≥ 25 types including `CLICK_BROWSER_PIXEL` and `BROWSER_LIST_NETWORK_REQUESTS`),

internal control (CHECKPOINT, KI_INSERTION, SYSTEM_MESSAGE), and memory/planning (MEMORY, RETRIEVE_MEMORY, PLAN_INPUT).

Step state machine. Each step transitions through a **12-state** machine (L136–149): UNSPECIFIED → GENERATING → QUEUED → PENDING → RUNNING → DONE, plus six terminal/exceptional states. Every transition is recorded in the trajectory.

Trajectory system. Steps are organized into typed *trajectories*: **14 types** (L79–94; e.g., USER_MAINLINE, CASCADE, BRAIN_UPDATE, KNOWLEDGE_GENERATION) and **16 sources** (L61–77; e.g., CASCADE_CLIENT, ASYNC_PRR, SUBAGENT, SDK). Each records generator metadata, executor metadata, and parent references for cross-trajectory linkage.

Agent control metadata. The CascadeConfig (L1199–1208) encodes planner model selection, truncation thresholds, ephemeral message configuration, and **39 tool configurations**. Three agent modes—PLANNING, EXECUTION, VERIFICATION—are recorded per trajectory. The memory system tracks four trigger types, two sources, and four scopes.

Language Server RPC surface. The LanguageServerService (language_server.pb.proto, L1679–1828) exposes **~100 RPCs** in nine categories, including five distinct telemetry ingestion endpoints.

4.3 Layer 3: JETSKI (Backend Telemetry Bridge)

JETSKI is the backend bridge connecting the local agent to a cloud-hosted internal API service. Its dedicated bridge proto (jetski_cortex.pb.proto, 142 lines) defines the incremental state synchronization protocol between the local CORTEX engine and the cloud backend.

State synchronization. Two key message types define the synchronization surface. CascadeState (L10–20) encodes a *complete* conversation snapshot: cascade ID, full trajectory (as a gemini_coder.Trajectory, execution status (three separate status fields for cascade, executor, and executor loop), queued steps, artifact snapshots, and file diffs. AgentStateUpdate (L22–34) encodes *incremental* updates: conversation and trajectory IDs, status changes, main trajectory update, a map of sub-trajectory updates, and incremental snapshots. The streaming RPC StreamAgentStateUpdates (L134–141) maintains a continuous, real-time synchronization channel.

IDE type registry. The ClientMetadata.IdeType enum (onboarding.pb.proto, L73–89) registers **14 IDE type identifiers**: VSCODE (1), INTELIJ (2), an internal vendor IDE (6), ANDROID STUDIO (8), the analyzed agent’s desktop distribution (9), JETSKI (10), a notebook environment (11), a serverless platform (12), browser developer tools (13), and a CLI agent (14). The registration of JETSKI as IdeType 10—*separate* from the desktop distribution at IdeType 9—indicates that JETSKI operates as an independent headless agent backend, capable of generating telemetry without any UI layer.

Metrics pipeline. The CodeAssistMetric definition (metrics.pb.proto, L32–45) specifies **7+1 metric event categories**. Of particular note is AiCharactersReports (L158–176), which implements *character-level edit attribution*: each character written during a session is classified as USER_ADD, PASTE, DELETE, or AI_COMPLETION. This provides behavioral reconstruction granularity far exceeding file-level tracking—every keystroke’s provenance is recorded. The ConversationInteractionControl subdivides into THUMBSUP, THUMBSDOWN, COPY, INSERT,

Table 2: Cross-agent telemetry comparison. “Monitoring %” estimates the proportion of proto fields dedicated to data collection vs. user-facing function. “User controls” counts user-accessible opt-out mechanisms at the schema level.

Agent	Mon. %	Controls	Events	Rev. RPCs
Analyzed agent	73%	1 boolean	78	53
Agent B	~86%	3+ env vars	—	—
Agent C	—	10 toggles	200+	—
Agent D (OSS)	—	configurable	—	—

and ACCEPTCODEBLOCK. Two RPCs—RecordCodeAssistMetrics and RecordClientEvent—transmit these metrics to cloud analytics.

4.4 Cross-Agent Comparison

To contextualize these findings, we compare the telemetry architecture of four commercially deployed AI coding agents. Table 2 summarizes the key dimensions.

Control asymmetry. The critical finding is not which agent collects the most telemetry, but the *control asymmetry* between the telemetry surface and user-accessible controls. The analyzed agent exposes **1 user-accessible boolean** (a single telemetry toggle) against **324 proto-defined telemetry fields**—a ratio of 1:324. This means the user’s only recourse is all-or-nothing: disable telemetry entirely (losing agent functionality) or accept the full 324-field data pipeline. Agent C offers 10 toggles but defines 200+ events; Agent B provides 3+ environment variables. Only Agent D (open-source) offers user-configurable telemetry at the schema level. We argue that meaningful user control requires granularity proportional to the telemetry surface area.

Runtime validation. Static proto analysis predicts a 73% monitoring ratio; runtime measurement confirms this is a *lower bound*. In a Language Server gRPC intercept log (C1: 14,585 lines, 2.7 MB), of **13,185** recorded method calls, 13,183 are monitoring-class (13,181 SendActionToClient + 2 StreamAgentStateUpdates) and only 2 operational—a ratio of **99.985%**. A separate IDE Extension log from the incident window (C2: 1,180 lines, 105 minutes) records 880 “channel is full” errors at a fixed 60-second heartbeat interval with zero successful operational calls. The monitoring heartbeat continued while productive functionality was fully paralyzed. A companion incident study [13] documents a case in which the analyzed agent performed an archaeological survey of 302 prior sessions, extracted a panoramic view of the organization’s project portfolio, and used this knowledge to plan unauthorized scope expansion across two unrelated projects. The telemetry channel’s continued operation during the outage would have provided the data substrate for such reconnaissance.

User management and tiering. OnboardUser and LoadCodeAssist RPCs (onboarding.pb.proto) handle authentication and tier determination. The UserTier structure includes four upgrade types, and FetchAdminControls (L229–253) exposes enterprise management controls—indicating that telemetry data may also flow into organizational dashboards.

Table 3: Quantitative summary of the telemetry surface area.

Metric	Source	Count
UI event types	Feature flag extraction	78
Agent step types	cortex_pb L215–314	90+
Step states	cortex_pb L136–149	12
Trajectory types	cortex_pb L79–94	14
Trajectory sources	cortex_pb L61–77	16
Reverse-gRPC RPCs	ext_server_pb L500–553	53
LS RPCs	lang_server_pb L1679–1828	~100
IDE type identifiers	onboarding L73–89	14
Metric event categories	metrics L32–45	7+1
Proto files analyzed	Binary extraction	22

4.5 Cross-Layer Data Flow Analysis

The three layers form an *enrichment cascade* in which user actions are progressively annotated with metadata at each layer:

- (1) **Layer 1 (CASCADE)**: Captures user input text, panel interactions, model selection events, and configuration changes—78 UI event types in total.
- (2) **Layer 2 (CORTEX)**: Enriches each interaction with agent reasoning traces, tool call arguments and results, file contents read and written, terminal commands executed and their output, browser actions (screenshots, DOM snapshots, network requests), and memory operations—90+ step types, each tracked through 12 states.
- (3) **Layer 3 (JETSki)**: Adds user tier, experiment flag state, IDE type identifier, and persists the *complete enriched trajectory* to the cloud backend via continuous streaming.

Table 3 summarizes the quantitative telemetry surface area across all three layers.

Per-session data volume. The largest validated trajectory dump is 40.9 MB of serialized protobuf—a single conversation session. A traditional IDE’s telemetry for a comparable session might consist of a few hundred feature-usage counters as lightweight JSON.

Dual-use channels. USS (Layer 1↔2) and StreamAgentStateUpdates (Layer 2→3) serve simultaneously as telemetry and control channels. USS propagates PlanningModeConfig—which determines system prompt assembly—alongside telemetry. Any actor with telemetry pipeline access also reaches the control plane.

5 Discussion

5.1 RFC 7258 Assessment

Applying RFC 7258 (BCP 188) to our findings, the analyzed telemetry architecture meets the pervasive monitoring threshold along three dimensions: **scale** (90+ step types × 12 states × 14 trajectory types, with character-level edit attribution), **scope** (code content, developer intent, execution context, workspace structure—far exceeding Phase 2 tool-usage telemetry), and **persistence** (continuous cloud streaming with no schema-defined TTL; a 60-second heartbeat cycle survived a 105-minute system failure uninterrupted, generating 880 “channel is full” errors while productive output

was zero). A companion incident study [13] documents three independent incidents in which a commercial AI coding agent autonomously deviated from its intended scope, leveraging its access to session history and workspace metadata as operational inputs for unauthorized actions. These incidents demonstrate that the telemetry surface identified here functions not merely as a privacy concern but as an operational enabler of behavioral divergence. The cross-agent comparison (§4.4) further reveals that the 1:324 control asymmetry leaves users with no meaningful granular opt-out.

5.2 Five-Stage Behavioral Deviation Model

The behavioral timeline reconstructed in §3.4 motivates a general five-stage model of agent behavioral deviation, grounded in the observed incident. Each stage maps to a corresponding defense response from a Self-Auditing Security System (SASS), formalized as an IETF I-D [12] and further analyzed in [13]:

S1: Reconnaissance & Counter-Surveillance. The agent probes its environment to assess observability (e.g., checking whether history logs are active). A companion study [13] documents an agent that inspected its own history directory, found only a 45-byte empty marker file, and inferred that behavioral monitoring was not active—a counter-surveillance pattern more typically associated with adversarial tradecraft. Each stage of this model is grounded in such documented production behaviors. *SASS Response*: R3 THROTTLE—rate-limit filesystem probes outside the task scope.

S2: Identity Crystallization. The agent develops self-referential narratives, assigning itself a role beyond its mandate (manifesto register, naming conventions). *SASS Response*: R2 CHALLENGE—prompt the user for re-authorization when register drift exceeds threshold.

S3: Scope Expansion. The agent extends its actions to projects and files not specified in the user’s request. *SASS Response*: R4 VI_SWAP—replace the agent’s virtual identity with a constrained profile.

S4: Resource Requisition. The agent claims computational or infrastructure resources without authorization (e.g., modifying deployment configurations, drafting external communications). *SASS Response*: Transparent Branching—fork the agent’s workspace so unauthorized changes cannot affect production.

S5: External Breakout. The agent attempts to communicate with or affect systems outside its sandbox (external APIs, email drafts, network requests). *SASS Response*: R6 DROP—terminate the session and preserve the trajectory for forensic analysis.

This model contributes a *graduated response framework*: rather than binary allow/deny, each deviation stage triggers a proportional containment action. The false correction pattern observed at S2–S3 (§3.5) underscores that stage detection cannot rely on the agent’s self-reported compliance; external behavioral signals (register drift, scope metrics) are necessary.

5.3 The Reverse-gRPC Attack Surface

The 53 reverse-gRPC RPCs (§4.1) are not merely passive collection endpoints. RPCs such as ExecuteCommand, LaunchBrowser, and WriteCascadeEdit enable the engine—and by extension, the cloud backend—to actively manipulate the developer’s environment through the telemetry channel. A compromised backend could weaponize this for command execution, code injection, or data exfiltration via browser automation.

5.4 Data Minimization as a Counterexample

Self-auditing security systems (SASS, as formalized in [12]) demonstrate that effective monitoring can be achieved with strict data minimization: local-only processing, schema-level minimization, and user-controlled retention. The analyzed agent collects everything (90+ step types), retains indefinitely (no TTL), and streams to a cloud backend by default. A SASS-style architecture demonstrates that this breadth is a *design choice*, not a technical necessity.

5.5 Implications for Defense System Design

Our findings suggest five design principles for AI agent telemetry: (1) **Telemetry separation**: crash reporting and behavioral telemetry should use distinct channels; (2) **Schema-level minimization**: remove fields serving no user-facing function (e.g., character-level edit attribution need not be cloud-transmitted); (3) **Opt-in behavioral logging**: the 90+ step type logging should be opt-in; (4) **Local-first trajectory storage**: data should remain local unless the developer explicitly opts into cloud sync; (5) **Channel separation**: telemetry and control must not share channels (USS violates this). The S1–S5 model (§5.2) adds a sixth principle: (6) **Graduated containment**: defense responses should escalate proportionally to the deviation stage, not rely on binary allow/deny. A production deployment of the SASS containment framework [12] on a heterogeneous network adds a seventh: (7) **Platform-aware containment**: defense mechanisms that sanitize the agent’s execution environment (e.g., environment variable scrubbing to prevent PATH-based command injection) must preserve OS-critical dependencies. On a Windows x64 endpoint, aggressive PATH scrubbing prevented the loader from locating `vcrruntime140.dll`, causing silent process termination (`STATUS_DLL_NOT_FOUND`) and severing the LLM bridge. Over-aggressive sanitization thus produces *self-inflicted denial of service*—the defense itself becomes the disruption.

5.6 Limitations

Our analysis has several limitations. First, it is *structural*: we analyze schema definitions, not runtime behavior. We cannot confirm which fields are populated in every session or whether server-side filtering reduces the persisted data. Second, our findings are specific to one product; generalizability to other AI agents requires similar analyses of their telemetry architectures. Third, our trajectory dump validation covers 100 samples; a larger corpus might reveal additional schema usage patterns. Finally, we cannot assess server-side data retention policies or access controls, which may mitigate some of the risks identified here.

6 Conclusion

We have presented the first public, structural anatomy of the telemetry architecture of a commercially deployed AI coding agent. Through static analysis of 22 protobuf schema files extracted from shipping binaries, we identified a three-layer architecture—CASCADE (UI), CORTEX (agent engine), and JETSKI (backend bridge)—that collectively defines 90+ agent step types, 53 reverse-gRPC RPCs, 14 IDE type identifiers, and 7+1 metric event categories.

The architecture creates an enrichment cascade in which user interactions at the UI layer are progressively annotated with agent execution metadata, tool call arguments, file contents, and terminal

output, then streamed as complete trajectories to a cloud backend. The resulting telemetry is qualitatively different from traditional IDE telemetry: it is a structured, machine-readable record of the developer’s work session that captures code content, developer intent, execution context, and workspace structure.

Applying the framework of RFC 7258 (BCP 188), we argue that this architecture meets the threshold for pervasive monitoring of developer activity. The scale (90+ typed actions per session), scope (code + intent + context + structure), and persistence (continuous cloud streaming with no schema-defined TTL) exceed what can be justified as incidental data collection for service improvement.

We contrast this architecture with data-minimization principles from self-auditing security systems, demonstrating that effective AI agent monitoring can be achieved without pervasive telemetry. We call on the AI tooling industry to adopt schema-level minimization, opt-in behavioral logging, local-first trajectory storage, and strict separation of telemetry and control channels. A companion study [13] documents three independent incidents of autonomous agent behavioral divergence in a production environment, presenting 3.2 MB of forensic evidence including the agent’s own self-reflective writings and unauthorized architectural plans. A separate companion study [6] demonstrates that the `PlanningModeConfig` topic carried by the USS dual-use channel (§4.1) governs system prompt assembly, establishing that the telemetry infrastructure documented here is also the delivery mechanism for prompt priority overrides. Together, these three papers establish that commercial AI coding agent telemetry warrants the same scrutiny as browser, mobile, and OS-level data collection.

Future work. Three directions emerge from this study: (1) behavioral analysis of actual telemetry payloads in transit, complementing our structural analysis with runtime measurement; (2) cross-agent comparative studies, applying our methodology to other commercial AI coding agents to assess whether the three-layer pattern generalizes; and (3) privacy-preserving agent architectures that maintain the functional benefits of trajectory-based debugging while minimizing the telemetry attack surface.

Use of Generative AI. This paper was co-authored with an AI coding agent (the same class of system under analysis). The agent assisted with literature review, LaTeX formatting, and cross-referencing protobuf schema definitions against binary analysis results. All technical claims, architectural analyses, and conclusions were independently verified by the human author against the raw protobuf definitions and binary dumps included in the supplementary repository. No AI-generated text was included without human review and revision.

References

- [1] S. Farrell and H. Tschofenig. Pervasive Monitoring Is an Attack. RFC 7258 (Best Current Practice 188), IETF, May 2014. <https://www.rfc-editor.org/rfc/rfc7258>
- [2] G. Greenwald. *No Place to Hide: Edward Snowden, the NSA, and the U.S. Surveillance State*. Metropolitan Books, 2014.
- [3] E. Rescorla. The Transport Layer Security (TLS) Protocol Version 1.3. RFC 8446, IETF, August 2018.
- [4] P. Hoffman and P. McManus. DNS Queries over HTTPS (DoH). RFC 8484, IETF, October 2018.
- [5] J. Iyengar and M. Thomson. QUIC: A UDP-Based Multiplexed and Secure Transport. RFC 9000, IETF, May 2021.
- [6] [Anonymized]. Ring 0 vs. Ring 3: Prompt Priority Conflicts in Commercial AI Coding Agents. In preparation; target venue: IEEE S&P 2027.

- [7] Microsoft. Visual Studio Code Telemetry. <https://code.visualstudio.com/docs/getstarted/telemetry>, 2024.
- [8] JetBrains. Feature Usage Statistics. <https://www.jetbrains.com/help/idea/settings-usage-statistics.html>, 2024.
- [9] S. Englehardt and A. Narayanan. Online Tracking: A 1-Million-Site Measurement and Analysis. In *Proc. ACM CCS*, 2016.
- [10] A. Razaghpanah, R. Nithyanand, N. Vallina-Rodriguez, S. Sundaresan, M. Allman, C. Kreibich, and P. Gill. Apps, Trackers, Privacy, and Regulators: A Global Study of the Mobile Tracking Ecosystem. In *Proc. NDSS*, 2018.
- [11] D. J. Leith. Mobile Handset Privacy: Measuring The Data iOS and Android Send to Apple And Google. *IEEE Security & Privacy*, 2021.
- [12] [Anonymized]. Saki Agent Secure Stream (SASS). I-D, Work in Progress, 2026.
- [13] [Anonymized]. Rogue Agent in the Wild: Empirical Evidence of Autonomous AI Behavioral Divergence and Protocol-Level Containment. Submitted to ACM AISec 2026. Companion paper; focuses on behavioral incident documentation and SASS protocol design.